

DAY 9

Bash Brace Expansion & Word Splitting – Detailed Notes

1. Brace Expansion

What is Brace Expansion?

Brace expansion is a Bash feature used to generate **multiple strings** from a **single expression**.

The basic syntax is:

```
{...}
```

It is particularly useful when you want to create multiple:

- Files
- Directories
- File names
- Paths
- Command arguments

without writing each one separately.

Example

```
echo {red,green,blue}
```

Output:

```
red green blue
```

The shell expands:

```
{red,green,blue}
```

into:

```
red
green
blue
```

before executing `echo`.

2. Types of Brace Expansion Lists

There are two important types:

1. **String lists**
 2. **Range lists**
-

2.1 String Lists

A **string list** contains arbitrary strings that don't need to follow a numerical or alphabetical sequence.

Syntax:

```
{string1,string2,string3}
```

Example:

```
echo {red,green,blue}
```

Result:

```
red green blue
```

Another example:

```
echo {cat,dog,bird}
```

Result:

```
cat dog bird
```

One of the most useful features is combining common text with a brace list.

For example:

```
echo file.{txt,pdf,doc}
```

file.txt file.pdf file.doc

Conceptually:



A **range list** generates an ordered sequence.

Numeric range

```
echo {1..5}
```

Output:

```
1 2 3 4 5
```

Alphabetical range

```
echo {a..e}
```

Output:

```
a b c d e
```

Uppercase also works:

```
echo {A..E}
```

Output:

```
A B C D E
```

5. Range Expansion with Steps

Bash also supports a step value.

For example:

```
echo {1..10..2}
```

Output:

```
1 3 5 7 9
```

Here:

```
1 → starting value
10 → ending value
2 → increment/step
```

You can also count backwards:

```
echo {10..1..2}
```

Result:

```
10 8 6 4 2
```

So the general form is:

```
{start..end..step}
```

6. Important Rule: No Spaces Inside Braces

This is a very important Bash syntax rule.

Correct:

```
{red,green,blue}
```

Incorrect:

```
{red, green, blue}
```

Why?

Brace expansion recognizes the comma-separated words as part of the brace expression. Spaces can change how the shell interprets the expression.

Remember:

```
{a,b,c}      ✓
{a, b, c}    ✗
```

Similarly:

```
{1..5}
```

not:

```
{1 .. 5}
```

7. Practical Example – Creating Multiple Directories

Suppose you want:

```
January  
February  
March  
April  
May
```

Instead of:

```
mkdir January  
mkdir February  
mkdir March  
mkdir April  
mkdir May
```

you can use:

```
mkdir {January,February,March,April,May}
```

Bash expands it into multiple arguments:

```
mkdir January February March April May
```

Then `mkdir` creates all of them.

8. Creating a Directory Structure

Brace expansion becomes particularly powerful when combined with nested braces.

For example:

```
mkdir -p project/{src,bin,docs}
```

creates:

```
project/  
├─ src/  
├─ bin/  
└─ docs/
```

You can go further.

For example:

```
mkdir -p project/{src,docs}/{day1,day2,day3}
```

can produce:

```
project/  
├─ src/  
│   ├─ day1/  
│   ├─ day2/  
│   └─ day3/  
└─ docs/  
    ├─ day1/  
    ├─ day2/  
    └─ day3/
```

This is why brace expansion is excellent for **rapid directory/file generation**.

9. Brace Expansion Is NOT a Loop

This is important conceptually.

When you write:

```
echo {1..5}
```

Bash is not executing a loop.

It is simply generating words:

```
1 2 3 4 5
```

The command effectively becomes:

```
echo 1 2 3 4 5
```

before `echo` executes.

So:

Brace expansion generates words; it does not execute commands repeatedly by itself.

10. Word Splitting

Now we move to a completely different concept.

Word splitting is a process where the shell takes the result of certain expansions and divides it into separate words.

It can occur after:

- Parameter expansion
- Command substitution
- Arithmetic expansion

when the result is **unquoted**.

11. Example of Word Splitting

Consider:

```
name="Sourya Dutta"
```

Now:

```
echo ${name}
```

The variable expands to:

```
Sourya Dutta
```

Because the expansion is **unquoted**, the shell can perform word splitting.

Conceptually:

```
${name}  
  ↓  
Sourya Dutta  
  ↓  
word splitting  
  ↓  
Sourya    Dutta
```

So the command receives two words/arguments.

12. Quoting Prevents Word Splitting

Now consider:

```
echo "${name}"
```

The expansion is inside double quotes.

Therefore, Bash treats the expanded value as one word:

```
"Sourya Dutta"
```

Conceptually:

```
"${name}"  
↓  
"Sourya Dutta"  
↓  
ONE WORD
```

This is one of the most important rules in shell scripting.

Remember :

```
${name}
```

→ potentially subject to word splitting.

```
"${name}"
```

→ treated as one word.

13. Why Quoting Is So Important

Suppose:

```
filename="my report.txt"
```

Then:

```
cat $filename
```

can be interpreted as:

```
cat my report.txt
```

which gives `cat` multiple arguments:

```
my  
report.txt
```

But:

```
cat "$filename"
```

passes the complete filename as one argument:

```
my report.txt
```

Therefore, a common Bash best practice is:

Quote variable expansions unless you intentionally want word splitting or another specific expansion behavior.

14. The IFS Variable

Word splitting is controlled by:

```
IFS
```

which stands for:

Internal Field Separator

Bash uses `IFS` to determine where words should be separated during word splitting.

You can inspect it with:

```
echo "${IFS@Q}"
```

However, because spaces, tabs, and newlines are difficult to see directly, another approach is:

```
printf '%q\n' "${IFS}"
```

The default `IFS` contains:

```
space  
tab  
newline
```

Conceptually:

```
IFS
|
├─ space
├─ tab
└─ newline
```

15. How IFS Controls Word Splitting

Suppose:

```
data="apple banana orange"
```

With the default `IFS`, Bash sees:

```
apple banana orange
   ↑       ↑
   separator
```

and splits it into:

```
apple
banana
orange
```

So:

```
"apple banana orange"
    ↓
  word splitting
    ↓
apple | banana | orange
```

16. Changing IFS

One powerful feature is that you can change `IFS`.

For example:

```
IFS=","
```

Now comma becomes a field separator for relevant word-splitting contexts.

Suppose:

```
data="apple,banana,orange"
```

With:

```
IFS=","
```

the data can be split into:

```
apple  
banana  
orange
```

Conceptually:

```
apple,banana,orange  
  ↓  
  IFS=","  
  ↓  
apple | banana | orange
```

This is useful when processing structured text.

17. Example: Processing Comma-Separated Data

Consider:

```
data="Linux,Networking,SOC"
```

Set:

```
IFS=", "
```

Now the comma acts as the separator when word splitting is applicable.

This technique can be useful for processing simple:

- CSV-like values
- Lists
- Configuration data
- Command output

However, for **real CSV files**, simply changing `IFS` is not sufficient because CSV has quoting, escaping, and other rules.

18. Important: `IFS` Is a Shell Variable

`IFS` itself is a shell parameter/variable.

You can inspect it:

```
printf '%q\n' "${IFS}"
```

You can temporarily modify it:

```
IFS=", "
```

But be careful when changing `IFS` in scripts because it affects word splitting behavior.

A safer pattern is often to limit the modified `IFS` to the specific operation where you need it.

19. Word Splitting and Command Substitution

Word splitting also matters when using command substitution.

Suppose a command produces:

```
hello world
```

For example:

```
result=$(some_command)
```

If you later use:

```
echo ${result}
```

the unquoted expansion may undergo word splitting.

But:

```
echo "${result}"
```

preserves the result as one word.

This is why quoting command-substitution results when storing/using text is often important.

20. Word Splitting and Parameter Expansion

Example:

```
name="Sourya Dutta"
```

Unquoted:

```
echo ${name}
```

The expanded value can be split according to `IFS`.

Quoted:

```
echo "${name}"
```

The entire value remains one word.

21. Word Splitting and Arithmetic Expansion

Arithmetic expansion is also one of the expansions included in the shell's processing model.

For example:

```
result=$((10 + 5))
```

produces:

```
15
```

Arithmetic results are generally numeric and don't usually create interesting word-splitting cases, but it is still important to know that **arithmetic expansion belongs to the expansion stage discussed here.**
